

Competitive Programming

Section 61: ICPC Contest Readiness

Competitive Programming Course

2026-07-10

Table of contents

1	Why the final section is about process	3
2	The contest board is shared memory	4
2.1	A useful status vocabulary	4
3	Triage is an estimate, not a promise	5
3.1	Worked triage example	5
4	The four-line pre-code contract	6
4.1	Trace: a useful handoff	6
5	One keyboard changes the workflow	7
6	Submission discipline	8
6.1	Scoreboard trace	8
7	Time boxes and recovery	9
8	Train the workflow in stages	10
8.1	Session 1: solo model sprint	10
8.2	Session 2: paired handoff	11
8.3	Session 3: team sprint	11
8.4	Session 4: full virtual and review	11
8.5	Advancement criteria	12
9	Three contest-operation algorithms	13
9.1	C++ notes	13
9.2	Python notes	13
10	Local judge workflow	14
11	Common mistakes	15
12	Before you submit	16
13	Exercises	17
14	Contest checklist	18

1 Why the final section is about process

At an ICPC contest, knowing an algorithm is necessary but incomplete. A team also loses time when two people silently work on the same idea, when a nearly finished solution blocks the only keyboard, or when a wrong answer is followed by random edits. The goal of this section is a process that preserves useful thinking under time pressure.

This is not a claim that every regional has identical rules. Read your event's rule sheet first. The local scoreboard drill defines the rule variant it uses. The transferable habit is to turn the written rule into state before writing code.

2 The contest board is shared memory

Keep one visible row per problem:

```
problem | owner | model claim | status | blocker | next test
```

The row should contain facts, not optimism. For example:

```
F | Noor | bipartite matching, left=jobs right=machines | proof | capacity? | duplicate a ma
```

An owner has responsibility for the next decision. A reviewer may challenge the proof, make a brute-force oracle, or prepare tests. This avoids the common failure mode where two partial solutions exist and neither is ready to submit.

2.1 A useful status vocabulary

Status	Meaning	Next action
scan	statement read once	write a model candidate or risk
model	reduction is stated	check constraints and proof
proof	invariant or exchange argument is clear	code
coding	one owner uses the keyboard	reviewer produces tests
test	sample plus adversarial cases agree	submit
parked	no progress inside the time box	record the exact blocker

parked is productive. It leaves a precise entry point for a teammate or for your future self. **stuck** is not a useful status because it says nothing about what information is missing.

3 Triage is an estimate, not a promise

During the first 10–15 minutes, every teammate scans every problem. Give each one a short card:

```
known tool: 0-1 BFS
work estimate: 25 min
risk: state graph may be too large
first proof question: is every transition cost 0 or 1?
```

Choose work by expected *accepted* progress, not by alphabetical order or by the problem that seems most interesting. A short implementation with a clear proof is often a better first solve than a familiar but bug-prone geometry task.

3.1 Worked triage example

Suppose a team sees these cards:

Problem	Estimate	Confidence	Risk
A	12 min	high	parsing
B	28 min	high	overflow
C	35 min	medium	endpoint convention
D	unknown	low	model absent
E	40 min	high	long implementation

One reasonable opening is A, B, and a second reading of C; park D. This is not a rigid schedule. If the second reading reveals a one-line model for D, update the board. The point is to make switching intentional rather than accidental.

4 The four-line pre-code contract

Before typing a medium or hard solution, the owner says or writes:

```
model: graph whose nodes are (cell, direction)
invariant: dist[v] is the best discovered distance when popped from Dijkstra
complexity:  $O((V + E) \log V)$ , within the stated limits
tests: one cell, unreachable target, a zero-cost turn
```

This takes under a minute. It exposes gaps early: an owner who cannot name the state probably should not start the implementation yet.

4.1 Trace: a useful handoff

Poor handoff:

```
"I tried G. It is probably DP."
```

Useful handoff:

```
"G is DP on prefixes. dp[i] is the minimum cost after the first i items.
The transition needs the last chosen index, so the one-dimensional state is
not enough. My counterexample is [3, 1, 3]. Please check whether a deque can
keep the candidate minima."
```

The second version transfers the model, the failed assumption, and the next experiment. A new owner can continue rather than reread the statement from zero.

5 One keyboard changes the workflow

When the keyboard is scarce, parallel work is still possible:

Role	Concrete job
owner	writes the smallest correct implementation
reviewer	traces the proof, samples, and boundary cases
tester	writes a brute force or hand-crafted adversarial inputs
scout	refreshes problem cards and looks for newly recognized reductions

Do not make roles permanent. Rotate after an accepted solution or a planned handoff. The important rule is that keyboard time belongs to a solution with a model and a test plan, not simply to the loudest unfinished task.

6 Submission discipline

For the common scoring format, a wrong answer before an eventual acceptance can add penalty. More generally, every submission costs attention and context. Submit when the following checklist is true:

1. The code matches the stated model and invariant.
2. A teammate or your written trace checks the sample independently.
3. You tested the smallest legal input and the most dangerous boundary.
4. Integer types, indexing convention, and output format are explicit.
5. You know what result a failure would falsify.

This does not mean waiting for absolute certainty. It means making the risk visible and testable.

6.1 Scoreboard trace

For the local rule, a team gains a solved problem only on its first accepted submission. The penalty for that problem is:

```
accepted minute + 20 * wrong attempts before that acceptance
```

Consider submissions for team 2 on problem 4:

Minute	Verdict	wrong attempts	solved	penalty contribution
18	W	1	0	0
31	W	2	0	0
47	A	2	1	87
52	W	2	1	87

The final wrong answer does nothing because the problem was already solved. Problem A turns this table into a reliable simulation.

7 Time boxes and recovery

A time box prevents an invisible hour of low-information work. A reasonable default is:

```
10--15 minutes: no model -> park with an exact blocker  
15--20 more minutes: model but no proof -> seek a reviewer  
20 minutes of debugging: build a tiny oracle or switch bug category
```

When a submission fails, classify before changing code:

```
parsing | model | proof | implementation | boundary | output format
```

For example, a failure only on $n = 1$ points toward a boundary or indexing issue; a failure on random tiny cases against a brute force points toward the model or implementation. This classification chooses the next test.

8 Train the workflow in stages

A five-hour virtual contest is valuable, but it is too large to diagnose a single weak habit. Train the behaviors separately, then combine them. Repeat this four-session loop with fresh problems from past ICPC-regionals or the external queue in `PRACTICE.md`.

Session	Format	Required artifact	What it isolates
1. Solo model sprint	75 min, 3 problems, no discussion	one four-line pre-code contract per problem	statement reading and algorithm selection
2. Paired handoff	90 min, 2 medium problems, one keyboard	a two-minute handoff note before every ownership change	state definition, proof communication, review
3. Team sprint	120 min, 4–5 problems under contest rules	shared board history and one classified test for each submission	triage, role rotation, submission discipline
4. Full virtual	target contest length, target team and machine rules	final board plus a post-contest log	integration under realistic fatigue and time pressure

The difficulty should rise across cycles, but the process does not change. A team that cannot maintain a correct board in a two-hour sprint will not repair that habit merely by extending the clock to five hours.

8.1 Session 1: solo model sprint

Give each person three problems that are individually within reach. For each problem, spend at most five minutes writing the model, invariant, complexity, and two tests. Code only the clearest one first. At the end, compare the contracts with the eventual editorials or an accepted solution, not just the final answers.

Record two numbers:

```
correct models before coding / problems read
minutes from first read to a precise model
```

These distinguish a recognition problem from an implementation problem. A fast wrong model is not progress; a slow but precise model tells you what to train.

8.2 Session 2: paired handoff

One person begins as owner. At a scheduled 20-minute mark, the partner must be able to continue using only the board entry and a spoken two-minute handoff:

```
state and invariant
what has been proved or tested
one counterexample, uncertainty, or blocker
the next action
```

The partner may reject the handoff if it lacks a state meaning or a falsifiable next step. That is useful feedback, not a social failure. Swap roles once even when the first owner could finish alone; practicing transfer is the point.

8.3 Session 3: team sprint

Use the actual number of keyboards and prohibit silent duplicate implementations. Every 25 minutes, pause for two minutes only to update the board: owner, status, blocker, and next test. Do not conduct a new group solve during the pause. This produces an audit trail of decisions without consuming the whole sprint in discussion.

For every submission, preserve one test input and label its purpose:

```
sample | smallest input | boundary | counterexample | random-oracle
```

After the sprint, inspect the first point where a wrong submission could have been caught. Add that input to the team's regression folder or notebook.

8.4 Session 4: full virtual and review

Run the full virtual with the target event's published rules. Keep the board, but do not pause the clock. Within 24 hours, upsolve each unsolved problem for a fixed 20 minutes before reading an editorial. Then write a one-line entry:

```
problem | missing observation | first evidence that would reveal it | next drill
```

For example, H | needed state (vertex, coupons_used), not vertex | two coupons reaching the same vertex | state-expansion sprint is actionable. H | hard DP is not.

8.5 Advancement criteria

Move from repeated sprints to frequent full virtuals only after two consecutive team sprints meet all of these observable conditions:

1. Every active problem has an owner, a concrete status, and a next test.
2. Every ownership change includes a model and a blocker or counterexample.
3. Every wrong submission is classified and gains a regression test or a written reason why no small test can expose it.
4. Each teammate both owns a solution and reviews a different solution.

These are process criteria, not a required solve count. A team can solve fewer problems and still improve sharply if its next virtual has fewer unforced mistakes and faster recoveries.

9 Three contest-operation algorithms

The exercises are intentionally familiar algorithmically. They test whether a team can deploy known tools accurately when the statement is dressed as a contest operation.

1. **Scoreboard Replay** is direct state simulation: each (`team`, `problem`) pair has a solved flag and a count of wrong attempts.
2. **Time-Boxed Schedule** is the deadline greedy from Section 9: retain a feasible set and remove its longest task when a deadline is exceeded.
3. **First-Round Assignment** is bipartite matching from Section 45: an edge means a team-mate can own a problem in the initial parallel round.

9.1 C++ notes

- Use a two-dimensional `vector` for scoreboard state; do not key it by a formatted string when team and problem ids are already bounded integers.
- Store penalties and elapsed time in `long long` even when the sample is small.
- `priority_queue<int>` is a max heap, exactly what the schedule greedy needs to remove the longest selected task.
- For matching, reset the right-side `seen` array for every DFS attempt.

9.2 Python notes

- A nested list of booleans and integers makes the scoreboard state visible.
- `heapq` is a min heap, so push negative durations to remove the longest selected duration.
- Use recursion for tiny matching instances only when the recursion depth is comfortably bounded; iterative alternatives are safer for very large input.

10 Local judge workflow

First run your stubs. They should fail until you implement them. Then verify the reference solutions and use the random tests as a regression check:

```
python3 sections/61_icpc_contest_readiness/check.py  
CP_TARGET=solution python3 sections/61_icpc_contest_readiness/check.py
```

For a solution with nontrivial logic, make one independent brute-force checker before trusting random tests. A generator that asks the same algorithm for the answer cannot reveal a shared mistake.

11 Common mistakes

- Treating a first scan as a commitment to solve the problem.
- Letting multiple people independently code the same task without a reason.
- Handing off only a feeling instead of the current model and counterexample.
- Updating a scoreboard after submissions to an already solved problem.
- Removing the shortest, rather than the longest, task in the deadline greedy.
- Reusing matching **seen** marks across separate augmenting-path searches.

12 Before you submit

```
[ ] The board says who owns this submission.  
[ ] I can state the model and invariant in one sentence each.  
[ ] The complexity uses the actual maximum constraints.  
[ ] A reviewer has traced a sample and a hostile tiny case.  
[ ] I know the next test if this receives a wrong answer.
```

13 Exercises

- A. Scoreboard Replay: simulate the stated ICPC-style scoring rules exactly.
- B. Time-Boxed Schedule: use a deadline greedy to maximize completed tasks.
- C. First-Round Assignment: model distinct initial owners as bipartite matching.
- D. Training-cycle retrospective: run one Session 2 or Session 3 drill from the table above. Preserve the board, one handoff note, and one test that would have prevented a real mistake. Convert the review into a single next drill rather than an unstructured list of topics.

14 Contest checklist

State the model, invariant, complexity, and next test before the keyboard. Record blockers precisely, park deliberately, and turn every post-contest mistake into a future test or notebook warning.